

Change scope of lambda *trailing-return-type*

Document #: P2036R3
Date: 2021-09-14
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[3 Potential Impact](#)

[No capture: `\[\]`.](#)

[init-capture: `\[a=expr\]` or `&a=expr`.](#)

[simple-capture: `\[b\]`, `&b`, `this`, or `\*this`.](#)

[capture-default with `&`.](#)

[capture-default with `=`.](#)

[4 Proposal](#)

[4.1 Parts of a Lambda](#)

[4.2 odr-used when not odr-usable](#)

[4.3 Feature-test macro](#)

[4.4 Implementation Experience](#)

[5 Wording](#)

[5.1 `\[expr.prim.id.unqual\]`](#)

[5.2 `\[expr.prim.lambda.capture\]`](#)

[6 Acknowledgements](#)

[7 References](#)

§ 1 Revision History

Since [\[P2036R2\]](#), wording changes.

Since [\[P2036R1\]](#), added feature test macro discussion and implementation experience.

Since [\[P2036R0\]](#), added wording and discussion of issues around getting the lookup correct.

§ 2 Introduction

There's a surprising aspect to the way that name lookup works in lambdas: it behaves differently in the *trailing-return-type* than it does in the lambda body. Consider the simple lambda implementing a counter:

```
auto counter1 = [j=0]() mutable -> decltype(j) {
    return j++;
};
```

The `decltype(j)` here is pointless (the deduced return type would be the same), but the real issue here is that it does not actually compile. That’s because the variable `j` we’re “declaring” in the init-capture isn’t actually “visible” yet (I’m using these terms somewhat loosely). The `j` in the body refers to the lambda’s “member” `j`, but the `j` in the *trailing-return-type* needs to find some outer `j` instead. Despite the capture being lexically closer to the lambda itself, and certainly far more likely to be the programmer’s intended meaning.

The best case scenario is that such code does not compile. The worst case scenario is that it does - because when it does compile, it means we had a situation like this:

```
double j = 42.0;
// ...
auto counter2 = [j=0]() mutable -> decltype(j) {
    return j++;
};
```

And now our lambda returns a `double` instead of an `int`.

This problem is most clear with *init-capture*, where we may actually be introducing new names. But it can show up in far more subtle ways with normal copy capture:

```
template <typename T> int bar(int&, T&&);           // #1
template <typename T> void bar(int const&, T&&);    // #2

int i;
auto f = [=](auto&& x) -> decltype(bar(i, x)) {
    return bar(i, x);
}
f(42); // error
```

Here, in the *trailing-return-type*, `x` refers to the parameter of the lambda, but `i` doesn’t refer to the lambda’s member (the lexically closest thing, declared implicitly via the `[=]`) but actually refers to the block scope variable, `i`. These are both `ints`, but the outer one is a mutable `int` while within the call operator of the lambda, is a `const int` (because the call operator is implicitly `const`). Hence the *trailing-return-type* gets deduced as `int` (via #1) while the expression in the body has type `void` (via #2). This doesn’t compile.

Another example arises from trying to write a SFINAE-friendly function composer:

```
template <typename F, typename G>
auto compose(F f, G g) {
    return [=](auto... args) -> decltype(f(g(args...))) {
        return f(g(args...));
    }
}
```

This implementation is buggy. The problem is the `f` and `g` from the body of the lambda are accessed as `const`, but from the *trailing-return-type* are not. Pass in a callable that’s intended to be non-`const`-

invocable (like, say, a `mutable` lambda), and we end up with a hard error when we finally instantiate the body.

For the *trailing-return-type* case, this problem only surfaces with *init-capture* (which can introduce new names) and any kind of copy capture (which may change the const qualification on some names). With reference capture (specifically either just `[&]` or `[&a]`), both the inner and outer uses of names are equivalent so there is no issue.

While it is possible (and quite easy) to produce examples that demonstrate this sort of different behavior, it's quite difficult to come up with examples in which this difference is actually desired and intended. I wrote a clang-tidy check to find any uses of problematic captures (those that are come from a copy capture or *init-capture*) and ran it on multiple code bases and could not find one. I would love to see a real world example.

This issue (the potentially-different interpretations of the same name in the *trailing-return-type* and lambda body) was one of (but not the only) reason that [\[P0573R2\]](#) was rejected. Consider this equivalent formulation of the earlier example, but with the abbreviated lambda:

```
template <typename T> int bar(int&, T&&);           // #1
template <typename T> void bar(int const&, T&&);    // #2

int i;
auto f = [=](auto&& x) => bar(i, x);
f(42); // still error
```

Here, we still error, for all the same reasons, because this lambda is defined to be equivalent to the previous one. But here, we only have one single `bar(i, x)` expression which nevertheless is interpreted two different ways.

As pointed out in that paper, it is quite common for users to “hack” this kind of lambda expression by using a macro that does the de-duplication for them. Such lambdas are broken if they use any kind of copy or *init-capture*. Or, more likely, somebody tried to write such a lambda, became confused when it didn't compile, flipped over a table, and then wrote it the long way.

This is one of those incredibly subtle aspects of the language today that are just needlessly confounding. It seems to me that whenever the meaning of an *id-expression* differs between the two contexts, it's a bug. I think we should just remove this corner case. It's also blocking reasonable future language evolution, and is likely a source of subtle bugs and preexisting user frustration.

§ 3 Potential Impact

Let's go through the various types of capture and see what the impact of this proposed change would be on usage and implementation.

§ No capture: `[]`

There is no capture, so there is no new thing to find. No change.

§ *init-capture*: `[a=expr]` or `[&a=expr]`

By the time we get to the *trailing-return-type*, we know the types of all the *init-capture* and we know whether the lambda is `mutable` or not, which means that we will know how to correctly interpret uses of `a` in the *trailing-return-type*. This will likely change the meaning of such code, if such code exists today. But note that such code seems fundamentally questionable so it's unlikely that much such code exists today.

§ *simple-capture*: `[b]`, `[&b]`, `[this]`, or `[*this]`

This is basically the same result as the *init-capture* case: we know the types by the time we get to the beginning of the *trailing-return-type*, so there are no issues determining what it should be.

With the reference capture cases (as well the *init-capture* spelling `[&a=a]`), there is actually no difference in interpretation anyway.

§ *capture-default with* `[&]`

With reference captures, there is no difference in interpretation between considering the capture and considering the outer scope variable. This paper would change nothing.

§ *capture-default with* `[=]`

This is the sad case. Specifically, in the case where:

1. We have a *capture-default* of `=`, and
2. We have a *trailing-return-type*, and
3. That *trailing-return-type* has an *id-expression* which is not otherwise covered by any other kind of capture, and
4. The use of that *id-expression*, if it appeared in the body, would be affected by the rule in 7.5.4.2 [\[expr.prim.id.unqual\]/3](#) (that is, it's not just `decltype(x)` but has to be either `decltype((x))` or something like `decltype(f(x))`), and
5. The lambda is not `mutable`, and
6. The variable is not `const`

Then we have a problem. First, let's go over the cases that are not problematic.

3. Eliminates cases like `[=, a]() -> decltype(f(a))`, which we know captures `a` by copy so we can figure out what the type of `a` would be when nominated in the body.
4. Eliminates cases like `[=]() -> x<decltype(a)>`, which actually have the same meaning in the body already.
5. Eliminates cases like `[=]() mutable -> decltype(f(a))`. Whether or not we end up having to capture `a`, the meaning of `f(a)` is the same in the body as it is in the *trailing-return-type*.

6. Eliminates cases like `[=]() -> decltype(g(c))` where `c` is, say, an `int const&`. Whether or not we end up having to capture `c`, the meaning of `g(c)` is the same in the body as it is in the *trailing-return-type*.

We're left with this pathological case:

```
int i;
[=]() -> decltype(f(i))
```

At this point, we do not know if we're capturing `i` or not. Today, this treats `i` as an lvalue of type `int` here. But with the proposed rule change, this *might* have to treat `i` as a `const` access, but only *if* we end up having to capture `i`:

```
auto f(int&) -> int;
auto f(int const&) -> double;

int i;

auto should_capture = [=]() -> decltype(f(i)) {
    return f(i);
};
auto should_not_capture = [=]() -> decltype(f(i)) {
    return 42;
};
```

Today, both lambdas return `int`. With the suggested change, the *trailing-return-type* needs to consider the capture, so we need to delay parsing it until we see what the lambda bodies actually look like. And then, we might determine that the lambda `should_capture` actually returns a `double`.

How can we handle this case?

1. We can, in this specific scenario (capture has an `=` and the lambda is `const`) just treat the *trailing-return-type* as token soup. The simplified rules for capture aren't based on return type [\[P0588R1\]](#) in any way, so this can work.
2. We can, in this specific scenario, just say that `i` is captured when used this way and that if it would not have been captured following the usual rules that the lambda is ill-formed.
3. We can say generally that any capturable entity in the *trailing-return-type* will behave as if it's captured (regardless of if it ends up being captured or not).

This paper suggests option 3. As with the rest of this paper, it is easy to come up with examples where the rules would change. Lambdas like the following would change meaning:

```
int i;
// previously returned int&, proposed returns int const&
// even though i is not actually captured in this lambda
auto f = [=](int& j) -> decltype((i)) {
    return j;
};
```

But it is difficult to come up with actual real-world examples that would break. And easy to come up with real-world examples that would be fixed by this change. The `lambda should_capture` would change to return a `double`, which seems more likely to be correct, and much more realistic an example than `f`.

§ 4 Proposal

This paper proposes that name lookup in the *trailing-return-type* of a lambda first consider that lambda's captures before looking further outward. We may not know at the time of parsing the return type which names actually are captured, so this paper proposes to treat all capturable entities as if they were captured.

That is, treat the *trailing-return-type* like the function body rather than treating it like a function parameter.

Such a change fixes the lambda in a way that almost certainly matches user intent, fixes the counter and compose lambdas presented earlier, and fixes all current and future lambdas that use a macro to de-duplicate the *trailing-return-type* from the body.

For the pathologically bad case (the use of a name in a *trailing-return-type* of a `const` lambda that nominates a non-`const` variable not otherwise accounted for in other lambda capture) that means we might have a lambda where we treat a name as captured when it might end up not actually having been captured - which would be a mistreatment in the opposite direction of the problem that this paper has been describing. This is unfortunate, but it's an especially strange corner case - one that's much more unlikely to appear in real code than the cases that this paper is trying to resolve.

§ 4.1 Parts of a Lambda

If we write out a lambda that has all the parts that it can have, they would be in the following order (most of these are optional):

1. *lambda-introducer*
2. *template-parameter-list*
3. *requires-clause* (#1)
4. *lambda-declarator*
 - a. *parameter-declaration-clause*
 - b. *decl-specifier-seq*
 - c. *noexcept-specifier*
 - d. *attribute-specifier-seq*
 - e. *trailing-return-type*
 - f. *requires-clause* (#2)
5. *compound-statement*

If we have a copy capture (whether it's a *simple-capture* or a *capture-default* of `=` or an *init-capture* that isn't a reference), the issue is we do not know what the type of a capture should be until we've seen whether the lambda is `mutable` or not (in the *decl-specifier-seq*).

What do we want to do about a case like this?

```
double x;  
[x=1](decltype((x)) y){ return x; }
```

There are four options for what this lambda could mean:

1. this is a lambda that takes a `double&` (status quo).
2. this is a lambda that takes an `int&` (lookup could be changed to find the *init-capture* but not do any member access transformation - even though this lambda ends up being not `mutable`)
3. this is a lambda that takes an `int const&` (would require lookahead, highly undesirable)
4. this is ill-formed

While there's a lot of motivation for the *trailing-return-type*, I have never seen anybody write this and do not know what the motivation for such a thing would be. (1) isn't very reasonable since the *init-capture* is lexically closer to use and it's just as surprising to find `::x` in the *parameter-declaration-clause* as it is in the *trailing-return-type*.

The advantage of (4) is that it guarantees that all uses of `x` in the *lambda-expression* after the *lambda-introducer* mean the same thing — we reject the cases up front where we are not sure what answer to give without doing lookahead. If motivation arises in the future for using captures in these contexts, we can always change the lookup in these contexts to allow such uses — rejecting now doesn't cut off that path.

This paper proposes (4).

Note that there are potentially *two* different *requires-clauses* in a lambda: one that is before the *decl-specifier-seq* and one that is after. Using a capture would be ill-formed in one but valid in the other:

```
double x;  
[x=1]  
  <decltype(x)* p> // ill-formed  
  requires requires {  
    *p = x;        // ill-formed  
  }  
  (decltype(x) q) // ill-formed  
  // now we know x is an lvalue of type int const  
  noexcept(noexcept(q+x)) // ok  
  -> decltype(q+x)         // ok  
  requires requires { q+x; } // ok  
{  
  return q+x;              // ok  
}
```

The status quo today is that all uses here are valid, and all of them save for the last one find `::x` (the `double`) — only in the lambda’s *compound-statement* does lookup find the *init-capture* `x` (the `int`).

§ 4.2 odr-used when not odr-usable

Davis Herring provides the following example:

```
constexpr int read(const int &i) {return i;}

auto f() {
    constexpr int value=3;
    return [=]() -> int(*)[read(value)] {
        static int x[read(value)];
        return &x;
    };
}
```

Today, this example is ill-formed (although no compiler diagnoses it) because `value` is odr-used in the *trailing-return-type*, but it is not odr-usable (6.3 [\[basic.def.odr\]/9](#)) there. It would be consistent with the theme of this paper (having the *trailing-return-type* have the same meaning as the body) to change the rules to allow this case. Such a rule change would involve extending the reach of odr-usable to include more of the parts of the lambda (but not default arguments) but making sure to narrow the capture rules (which currently are based on odr-usable) to ensure that we don’t start capturing more things.

I’m wary of such a change because I’m very wary of touching anything related to ODR. Especially because in an example like this, we could easily make `value` not odr-used here (either by making `value` `static` or by changing `read` to not take by reference).

§ 4.3 Feature-test macro

The change this paper suggests doesn’t merit a feature test macro. If you had to support both the old and new versions, you would just directly write the code the old way that would get you the behavior you wanted with the new way. Writing two different *trailing-return-types* or *noexcept-specifiers* doesn’t seem like it would provide any value - just write the one that always works.

§ 4.4 Implementation Experience

None, but the behavior changes suggested here don’t require any kind of parsing heroics. We still have everything we need to know at the time that we’re parsing the *trailing-return-type*, it’s just that there’s a new scope that is looked in first. No implementors have reported any concerns.

§ 5 Wording

This wording is based on the working draft after Davis Herring’s opus [\[P1787R6\]](#) was merged (i.e. [\[N4878\]](#)).

The wording strategy here is as follows. We have the following scopes today:

- *lambda-introducer*
- *template-parameter-list*
- *requires-clause* (#1)
- *lambda-declarator*
 - *parameter-declaration-clause* (function parameter scope)
 - *decl-specifier-seq*
 - *noexcept-specifier*
 - *attribute-specifier-seq*
 - *trailing-return-type*
 - *requires-clause* (#2)
 - *compound-statement* (block scope)

We have to move the *init-capture* to inhabit the function parameter scope, making sure to still reject cases like:

- `[x=1](int x){}` (currently rejected by 6.4.3 [\[basic.scope.block\]/2](#), the *init-capture* targets the *compound-statement* and the function parameter targets the parent of that)
- `[x=1]{ int x; }` (currently rejected by 6.4.1 [\[basic.scope.scope\]/4](#), the two declarations of `x` potentially conflict in the same scope)

We then have to change the `[expr.prim.id.unqual]` rule such that if an *unqualified-id* names a local entity from a point `S` within a lambda-expression, we first consider the point `S'` that is within the *compound-statement* of that innermost lambda. If, from `S'`, some intervening lambda (not necessary the innermost lambda from `S'`) would capture the local entity by copy then:

- if `S` is in that innermost capturing lambda's function parameter scope but not in the *parameter-declaration-clause*, then we do the class member access transformation.
- otherwise, we say the access is ill-formed.

To clarify:

```
int x;
[=]<decltype(x)* p> // error: unqualified-id names a local entity that would be
    // captured by copy
    // but not from the function parameter scope
(decltype(x) y) // error: unqualified-id names a local entity that would be
    // captured by copy
    // from within the function parameter scope, but it's in the
    // parameter-declaration-clause
-> decltype((x)) // ok: unqualified-id names a local entity that would be
    // captured by copy
    // in the function parameter scope, transformed into class
    // access. Yields int const&.
{
    return x; // ok: lvalue of type int const
};
```

```
int j;
[=]() {
    [<decltype(j)* q> // ok: the innermost lambda that would capture j by copy is the
        outer lambda
        // and we are in the outer's lambda's function parameter scope,
        this is int*
    (decltype((j)) w) // ok: as above, 'w' is a parameter of type int const&
    {}
};
```

§ 5.1 [expr.prim.id.unqual]

Change 7.5.4.2 [\[expr.prim.id.unqual\]](#)/3, including adding bullet points to make it clear what branch each case refers to and a drive by fix the issue Tim Song pointed out [here](#):

- 3 The result is the entity denoted by the *unqualified-id* ([basic.lookup.unqual]). If the *unqualified-id* appears in a *lambda-expression* at program point P and the entity is a local entity or a variable declared by an *init-capture* ([expr.prim.lambda.capture]), then let S be the *compound-expression* of the innermost enclosing *lambda-expression* of P. ~~and If naming it the entity~~ from outside of an unevaluated operand within ~~the scope where the unqualified-id appears~~ S would ~~result in some intervening lambda-expression capturing it by copy~~ refer to an entity captured by copy in some intervening *lambda-expression* ([expr.prim.lambda.capture]), then let E be the innermost such *lambda-expression*, and:
- (3.1) — If P is in E's function parameter scope but not its *parameter-declaration-clause*, then the type of the expression is the type of the class member access expression ([expr.ref]) naming the non-static data member that would be declared for such a capture in the closure object of ~~the innermost such intervening lambda-expression~~ E.
- [Note 3: If ~~that lambda-expression~~ E is not declared *mutable*, the type of such an identifier will typically be *const* qualified. — end note]
- (3.2) — Otherwise (if P either precedes E's function parameter scope or is in E's *parameter-declaration-clause*), the program is ill-formed.

Otherwise, the type of the expression is the type of the result.

Extend the example in 7.5.4.2 [[expr.prim.id.unequal](#)]/3 to demonstrate this rule:

[Example 1:

[illegible]

```

+   decltype((x)) y2 = y1;      // y2 has type float const&
+   decltype(r) r1 = y1;        // r1 has type float&
+   decltype((r)) r2 = y2;      // r2 has type float const&
+   return y2;
+   };

+   [=]<decltype(x) P>{};        // error: x refers to local entity but
+   precedes the
+   // lambda's function parameter scope
+   [=](decltype((x)) y){};      // error: x refers to local entity but is
+   in lambda's
+   // parameter-declaration-clause
+   [=]{
+   +   [<decltype(x) P>{};        // ok: x is in the outer lambda's function
+   +   parameter scope
+   +   [&](decltype((x)) y){};    // ok: lambda takes a parameter of type
+   +   float const&
+   +   [x=1](decltype((x)) z){};  // error: x refers to init-capture but is
+   +   in lambda's
+   +   // parameter-declaration-clause
+   +   };
+   }

```

- end example]

§ 5.2 [expr.prim.lambda.capture]

Change 7.5.5.3 [expr.prim.lambda.capture]/6:

- 6 An *init-capture* inhabits the **function parameter** scope of the *lambda-expression*'s **~~compound-statement~~ parameter-declaration-clause**. An *init-capture* without ellipsis behaves as if it declares and explicitly captures a variable of the form [...]

And extend the example to demonstrate this usage (now we do have an `i` in scope for `decltype(i)` to find):

```

int x = 4;
auto y = [&r = x, x = x+1]()->int {
    r += 2;
    return x+2;
    }; // Updates ::x to 6, and
      initializes y to 7.

auto z = [a = 42](int a) { return 1; }; // error: parameter and
      local variable have the same name

+ auto counter = [i=0]() mutable -> decltype(i) { // ok: returns int
+   return i++;
+ };

```

Our earlier bad examples of *init-capture* should still be rejected:

- `[x=1](int x){}` is now rejected by 6.4.1 [basic.scope.scope]/4, since we now have two declarations of `x` in the function parameter scope of the lambda.

- `[x=1]{ int x; }` is now rejected by 6.4.3 [basic.scope.block]/2, since the declaration `int x` targets the block scope of the *compound-statement* of the lambda and `x=1` is a declaration whose target scope is the function parameter scope, the parent of that *compound-statement*.

Basically, we've just swapped which rule rejects which example, but both examples are still rejected.

§ 6 Acknowledgements

Thanks to Davis Herring for all of his work, just in general. Thanks to Tim Song for help understand the rules.

§ 7 References

[N4878] Thomas Köppe. 2020-12-15. Working Draft, Standard for Programming Language C++.

<https://wg21.link/n4878>

[P0573R2] Barry Revzin, Tomasz Kamiński. 2017-10-08. Abbreviated Lambdas for Fun and Profit.

<https://wg21.link/p0573r2>

[P0588R1] Richard Smith. 2017-11-07. Simplifying implicit lambda capture.

<https://wg21.link/p0588r1>

[P1787R6] S. Davis Herring. 2020-10-28. Declarations and where to find them.

<https://wg21.link/p1787r6>

[P2036R0] Barry Revzin. 2020-01-12. Changing scope for lambda trailing-return-type.

<https://wg21.link/p2036r0>

[P2036R1] Barry Revzin. 2021-01-13. Changing scope for lambda trailing-return-type.

<https://wg21.link/p2036r1>

[P2036R2] Barry Revzin. 2021-07-23. Changing scope for lambda trailing-return-type.

<https://wg21.link/p2036r2>